

Stage-2 Variants — TFLite Benchmark Files & Commands

Companion to the TinyCNN benchmark notes. Same script, same output layout, same INA219 measurement chain.

1. Project Location

The project is located at:

```
/media/shashwat/A431-E4641/ronin_yolov26
```

The Python virtual environment used is `~/ronin_env`. Activate it, then enter the project directory:

```
source ~/ronin_env/bin/activate
cd /media/shashwat/A431-E4641/ronin_yolov26
```

2. Main Benchmark File

Unchanged — the same script benchmarks these models:

```
tools/run_tflite_benchmark.py
```

Every existing `--which` name, flag and default behaves exactly as before. One line changed inside the script's model dispatch; five new model names were added.

3. Input Data

The benchmark uses the same sequence as the TinyCNN run:

```
--root_dir data/seen_subjects_test_set
--sequence a000_11
```

4. TFLite Model and Stage-2 Variants

All six targets share the same backbone, `models_tflite/yolo26.tflite`. They differ only in the stage-2 corrector applied after it. Selected with `--which`:

--which	Stage 2	Runtime needed
yolo26	none (backbone only)	tflite
yolo26_A_rf	Random Forest	tflite + sklearn
yolo26_B_ridge	Ridge / linear	tflite + numpy
yolo26_C_ema	Causal EMA	tflite + numpy
yolo26_D_mlp	Small MLP	tflite + numpy
yolo26_E_tcn	Causal TCN	tflite + numpy

PyTorch is **not** required on the Pi. Only yolo26_A_rf needs scikit-learn, which is already installed for the existing Random Forest benchmark.

5. Files to Copy

Create stage2_models/ at the project root, beside the existing models_tflite/ and rf_yolo26/ directories:

File	Size	Needed for
B_ridge_corrector.npz	3.3 KB	yolo26_B_ridge
C_ema_corrector.npz	2.3 KB	yolo26_C_ema
D_mlp_corrector.npz	26 KB	yolo26_D_mlp
E_tcn_corrector.npz	55 KB	yolo26_E_tcn
A_rf_corrector.npz	2.4 KB	yolo26_A_rf
A_rf_corrector_sklearn.joblib	128 MB	yolo26_A_rf

The four numpy variants total 87 KB. The correction gain and clip bound are stored inside each .npz, so --rf_alpha and --rf_clip are not used for these targets. Use --stage2_dir if the files are placed elsewhere.

6. Benchmark Commands

Smoke test first. Run one iteration of the simplest and the most complex corrector to confirm the files are found and the numpy path executes on ARM:

```
python3 tools/run_tflite_benchmark.py --which yolo26_C_ema --runs 1 --root_dir
data/seen_subjects_test_set --sequence a000_11

python3 tools/run_tflite_benchmark.py --which yolo26_E_tcn --runs 1 --root_dir
data/seen_subjects_test_set --sequence a000_11
```

Each should print a line ending neural=... rf=... total=... and write a CSV under bench_results_tflite/.

Regression check. Confirm the existing workflow is untouched:

```
python3 tools/run_tflite_benchmark.py --which tinycnn --runs 1 --root_dir
data/seen_subjects_test_set --sequence a000_11
```

Full 50-run benchmark, all five variants plus the backbone baseline. The backbone runs first so stage-2 overhead can be isolated by subtraction:

```
for M in yolo26 yolo26_A_rf yolo26_B_ridge yolo26_C_ema yolo26_D_mlp yolo26_E_tcn; do
  python3 tools/run_tflite_benchmark.py --which $M --runs 50 --root_dir data/seen_subjects_test_set
  --sequence a000_11
done
```

If the shell variable is lost when pasting, run the six commands individually with the model name written out.

7. Output Files

Results are written to `bench_results_tflite/`, following the existing pattern:

- `<model>_benchmark_results.csv` — per-run results
- `<model>_benchmark_summary.txt` — per-model summary
- `benchmark_summary.txt` — overall summary

CSV columns are unchanged:

```
model, run, voltage_v, current_a, power_w, energy_j, sample_time_ms,
seq_time_ms, neural_ms, rf_ms, total_ms, mse_x, mse_y, returncode
```

For these targets the `rf_ms` column holds the stage-2 time: feature construction, corrector inference, and the correction itself.

8. Checking the Results

```
wc -l bench_results_tflite/yolo26_C_ema_benchmark_results.csv
cat bench_results_tflite/yolo26_C_ema_benchmark_summary.txt
cat bench_results_tflite/benchmark_summary.txt
```

9. Hardware Measurement

An INA219 is used for voltage/current/power measurement during the benchmark, exactly as in the TinyCNN run. Recorded measurements: Voltage (V), Current (A), Power (W), Energy (J).

10. Expected Stage-2 Overhead

Measured on x86 over a 7,117-sample trajectory, with a Pi estimate scaled by 5×. That factor is derived from the existing measurement — the Random Forest's published Pi overhead is 0.268 ms/sample against an x86 cost of 0.0555 ms/sample.

Variant	x86 (ms/sample)	Pi estimate (ms/sample)
B_ridge	0.0044	~0.02
D_mlp	0.0048	~0.02
C_ema	0.0078	~0.04
E_tcn	0.0203	~0.10
A_rf	0.0555	~0.28

All five sit well inside the roughly 9.3 ms left over by the backbone under the 50 ms update budget, so none should threaten real-time operation.

11. Notes Before Quoting Results

Accuracy columns are provisional. These correctors were fitted on a dataset that included the unseen test set, and from a backbone checkpoint that could not be identified. Latency, power and energy are architecture-driven and will hold, but `mse_x` and `mse_y` should not be published until the correctors are retrained. The `.npz` files are regenerated automatically by the same training command, so re-copy them at that point.

The TFLite backbone is not the final checkpoint. `yolo26.tflite` was exported from epoch 35; the finalized model is epoch 43. Timing and power are unaffected — identical architecture, identical compute — but the accuracy columns again reflect the older weights.